

Ghana AI Talent Accelerator

12-Week Curriculum & Pacing Guide

Speedykom Group GmbH

Curriculum Progression & Prerequisites

Learning Path Overview

Module 00: Why Python?

↓

Module 01: Setup (uv + VS Code)

↓

Module 02: Version Control (Git/Github)

↓

Module 03: Python Basics (Syntax, Data Structures)

↓ (Hard prerequisite)

Module 04: Data Analysis (Pandas)

↓ (Hard prerequisite)

Module 05: Machine Learning (scikit-learn)

↓

Module 06: Deep Learning (PyTorch + Google Colab)

↓

Module 07: Agentic AI (LLM APIs, Advanced)

Hard vs. Soft Prerequisites

Hard Prerequisites (MUST KNOW)

Students cannot progress without mastering these:

Blocking Module	Blocks	Why	Time to Master
Module 03 (Python Basics)	Module 04, 05, 06, 07	All subsequent modules use Python syntax	2-3 weeks
Module 04 (Pandas)	Module 05, 06, 07	Can't do ML without data manipulation	2-3 weeks
Module 05 (ML)	Module 06	Must understand traditional ML before deep learning	2-3 weeks

Soft Prerequisites (STRONGLY RECOMMENDED)

- Module 01 (Setup): Technically can skip, but won't have working environment
- Module 02 (Version Control): Needed for collaboration, not core to learning Python/AI

Knowledge Gates: Where Students Get Stuck

Gate 1: Python Mindset (Module 03)

Blocker: Students write JavaScript-style Python code **Symptom:** Using `undefined` patterns, expecting object-oriented defaults **Solution:** Extra exercises on “Pythonic” thinking (list comprehensions, dynamic typing)

Gate 2: Data Manipulation (Module 04)

Blocker: Confusion between Series, DataFrames, and NumPy arrays **Symptom:** Type errors when working with `.loc[]` vs. `[]` indexing **Solution:** Visual examples showing data structure transformations

Gate 3: ML Intuition (Module 05)

Blocker: Don't understand train/test split or why it matters **Symptom:** Training model on full dataset, overfitting is “good” to them **Solution:** Walk through real dataset overfitting example visually

Gate 4: Deep Learning (Module 06)

Blocker: GPU concepts and computational graphs are abstract **Symptom:** Running code on CPU when it should run on GPU; code is 10x slower **Solution:** Google Colab demo showing GPU usage; explain why it matters

Estimated Pacing

Total Program: 12-16 weeks (full-time) or 6-8 months (part-time)

Module	Duration	Difficulty	Type
00	1 week	Easy	Conceptual
01	1-2 weeks	Easy	Setup/Configuration
02	1 week	Easy	Workflow
03	3-4 weeks	Medium	Foundational
04	3 weeks	Medium	Foundational
05	3 weeks	Medium-Hard	Foundational
06	2-3 weeks	Hard	GPU/Compute-heavy
07	2 weeks	Hard	Project-based

Prerequisite Knowledge from Before Course

Required (Module 00 validates this)

- Basic programming logic (loops, conditionals)
- Understanding of web development stacks (HTML, JavaScript, APIs)
- Comfort with terminal/command line
- Basic math (algebra, probability)

Nice to Have

- Database concepts (SQL-like thinking helps with Pandas)
- Linear algebra (helps with ML/DL intuition)
- Statistics background (ML is applied statistics)

Adaptation Paths (By Student Background)

Path A: MERN Developer (Main Target)

- **Strengths:** Already knows web dev, APIs, async programming
- **Challenges:** JavaScript thinking gets in the way
- **Adaptation:** Extra emphasis on Python mindset (Module 03), show web-to-ML career path
- **Time:** 12-14 weeks

Path B: Backend/Systems Developer (Go, Rust, Java)

- **Strengths:** Understands typed thinking, systems concepts, performance
- **Challenges:** May resist “dynamic” Python; over-optimize early
- **Adaptation:** Show where Python’s dynamism is a feature, not a bug
- **Time:** 10-12 weeks

Path C: Absolute Beginner

- **Strengths:** No bad habits, clean slate
- **Challenges:** Needs more explanation of “why” before “how”
- **Adaptation:** Extra basic examples, slower pacing on fundamentals
- **Time:** 16-20 weeks

Success Metrics Per Module

Module 03 (Python Basics)

Write list comprehension without thinking Explain why `[1, 2, 3]` is mutable but `(1, 2, 3)` isn't Use f-strings naturally

Module 04 (Data Analysis)

Load CSV, filter rows, group by column Explain `.loc[]` vs. `.iloc[]` difference Spot obvious data quality issues

Module 05 (Machine Learning)

Build train/test split pipeline Explain what overfitting looks like Choose appropriate algorithm for problem

Module 06 (Deep Learning)

Build neural network and understand each layer Train on GPU without thinking Read PyTorch error messages and debug

Module 07 (Agentic AI)

Build simple AI agent using LLM API Understand function calling and prompts Deploy and test in real scenario

12-Week Curriculum Pacing Guide

Overview

This 12-week intensive program transforms MERN developers into Python/AI engineers. Structure: 3-4 hours/week in-class + 10-15 hours self-study.

Week-by-Week Breakdown

Week 1: Why Python & Environment Setup (4-5 hours)

Notebooks: - `python-vs-other-languages.qmd` (2 hours) — Why Python matters - `setting_up.qmd` (2-3 hours) — Install uv, VS Code, create first project

Learning Objectives: - Understand why Python dominates AI/Data Science - Install development environment successfully - Create and run first Python project

In-Class: - Discuss language tradeoffs - Live setup walkthrough - Troubleshoot environment issues

Self-Study: - Complete setup on personal machine - Run test script to verify installation

Week 2: Version Control (3-4 hours)

Notebooks: - `Git_Github.qmd` (3-4 hours) — Git fundamentals and GitHub workflow

Learning Objectives: - Understand version control purpose - Use Git for individual and collaborative work - Push/pull from GitHub confidently

In-Class: - Git workflow walkthrough - Pair programming setup - Merge conflict resolution demo

Self-Study: - Practice Git locally - Fork and contribute to sample repo - Create personal portfolio repo

Week 3: Python Basics Part 1 — Variables & Data Structures (4-5 hours)

Notebooks: - `intro_python1.qmd` (4-5 hours) — Mindset shift, Big Four data structures

Learning Objectives: - Shift from JavaScript to Python thinking - Master lists, tuples, dictionaries, sets - Use negative indexing and slicing fluently - Write Pythonic code (not JavaScript in Python)

Key Concepts: - Indentation (not curly braces) - Dynamic typing - The Big Four data structures - F-strings

In-Class: - Live Python REPL coding - JS vs. Python comparisons - Common student mistakes walkthrough - Group exercises on data structures

Self-Study: - 10+ exercises on each data structure - Build shopping cart/inventory system with lists/dicts - Practice negative indexing until natural

CRITICAL GATE: Students who don't master this week will struggle for rest of course.

Week 4: Python Basics Part 2 — Functions & Scope (4-5 hours)

Notebooks: - intro_python2.qmd (4-5 hours) — Functions, arguments, scope, best practices

Learning Objectives: - Write clean, reusable functions - Understand variable scope (local vs. global) - Use default arguments and `*args/**kwargs` - Document code with docstrings

Key Concepts: - Function definition and calling - Parameters and arguments - Return values - Scope rules (LEGB) - Lambda functions

In-Class: - Function design patterns - Debugging scope issues - Code organization best practices - Refactoring exercises

Self-Study: - Write 15+ functions (calculator, string processors, list filters) - Understand scope with visual debugger - Document functions with Google-style docstrings

Week 5: Python Basics Part 3 — Comprehensions & Advanced (4-5 hours)

Notebooks: - intro_python3.qmd (4-5 hours) — Comprehensions, generators, iterators, Pythonic patterns

Learning Objectives: - Write list/dict/set comprehensions naturally - Understand generators and lazy evaluation - Use iterators effectively - Recognize and write Pythonic code patterns

Key Concepts: - List comprehensions `[x for x in ...]` - Dict/set comprehensions - Generator expressions - The `zip()`, `map()`, `filter()` functions - Pythonic idioms

In-Class: - Comprehension practice (lots of examples) - Generator walkthrough - Comparing comprehensions vs. loops - Code style review

Self-Study: - Convert 20+ loops to comprehensions - Write generators for common tasks - Read and understand Pythonic code samples

SUCCESS METRIC: If 80%+ of students naturally use comprehensions and avoid explicit loops, you're ready for Pandas.

Week 6: Data Analysis Part 1 — DataFrames & Series (5-6 hours)

Notebooks: - intro_pandas1.qmd (5-6 hours) — DataFrames, Series, indexing, basic operations

Learning Objectives: - Understand DataFrames as “spreadsheets in Python” - Load and explore data (shape, dtypes, head, info) - Select data using .loc[], .iloc[], and column selection - Perform basic operations (sort, unique, value_counts)

Key Concepts: - Series (1D) vs. DataFrame (2D) - Indexing: integer position (.iloc) vs. labels (.loc) - Boolean indexing - Data types and type conversion - Loading data (CSV, JSON, Excel)

In-Class: - Live data loading demos - .loc[] vs. .iloc[] drills (confuses everyone!) - Real Ghanaian datasets walkthrough - Debugging data issues

Self-Study: - Load 5+ datasets (CSV, JSON, etc.) - Explore each: shape, dtypes, first/last rows - Practice .loc[], .iloc[] until automatic - Answer “what % of data is missing?” questions

COMMON MISTAKE: Confusing .loc[] (labels) with .iloc[] (positions). Heavy drilling needed.

Week 7: Data Analysis Part 2 — Transformation & Cleaning (5-6 hours)

Notebooks: - intro_pandas2.qmd (5-6 hours) — Selection, filtering, transformation, cleaning

Learning Objectives: - Filter rows based on conditions - Create new columns from existing ones - Handle missing data (drop, fill, interpolate) - Rename and reorder columns - Work with string columns

Key Concepts: - Boolean indexing (df[df['column'] > value]) - Assigning new columns - .fillna(), .dropna(), .interpolate() - String methods (.str.upper(), .str.contains(), etc.) - .apply() for custom transformations

In-Class: - Data cleaning walkthroughs (real messy data!) - Boolean indexing practice - Column creation exercises - “Spot the data quality issue” games

Self-Study: - Clean 3 real datasets (handle nulls, outliers, types) - Create derived columns (age from birthdate, salary band from salary) - Practice filtering until it’s automatic - Use .apply() on string columns

Week 8: Data Analysis Part 3 — Grouping & Aggregation (5-6 hours)

Notebooks: - intro_pandas3.qmd (5-6 hours) — Groupby, aggregation, pivot tables, merging

Learning Objectives: - Group data by categories (groupby) - Aggregate (sum, mean, count, custom) - Create pivot tables and cross-tabs - Merge/join multiple datasets - Understand database-like operations in Pandas

Key Concepts: - .groupby() mechanics - Aggregation functions (.agg(), .sum(), .mean()) - Pivot tables - Merges (inner, outer, left, right) - Setting and resetting index

In-Class: - Groupby walkthroughs with real data - Pivot table demos - Join/merge strategy discussions - Debugging groupby confusion

Self-Study: - Groupby: answer “What’s average revenue by region?” style questions - Create pivot tables from raw data - Merge 2-3 datasets together - Practice until groupby is intuitive

SUCCESS METRIC: Students can answer any “group and aggregate” question without looking up syntax.

Week 9: Machine Learning Part 1 — Concepts & Supervised Learning (5-6 hours)

Notebooks: - intro_ML1.qmd (3 hours) — ML concepts, train/test split, evaluation - intro_ML2.qmd (2-3 hours) — Supervised learning (classification, regression)

Learning Objectives: - Understand train/test split philosophy and why it matters - Build classification models (Logistic Regression, Decision Trees) - Build regression models (Linear Regression) - Evaluate models (accuracy, precision, recall, RMSE) - Identify and diagnose overfitting

Key Concepts: - Train/test split (why 80/20?) - Supervised vs. unsupervised learning - Classification vs. regression - Feature preprocessing (scaling, encoding) - Model evaluation metrics

In-Class: - Train/test philosophy deep dive (show real overfitting example!) - Live model training - Metric interpretation discussion - Overfitting vs. underfitting visual demos

Self-Study: - Build 3 classification models (different datasets) - Build 2 regression models - Tune hyperparameters - Identify overfitting in your models

Week 10: Machine Learning Part 2 — Unsupervised & Evaluation (5-6 hours)

Notebooks: - intro_ML3.qmd (5-6 hours) — Unsupervised learning, clustering, PCA, model selection

Learning Objectives: - Perform clustering (K-Means, Hierarchical) - Understand dimensionality reduction (PCA) - Apply multiple metrics to evaluate clustering - Choose appropriate algorithms for problems - Use cross-validation for robust evaluation

Key Concepts: - Clustering (K-Means, DBSCAN, Hierarchical) - PCA (dimensionality reduction) - Elbow method (finding K) - Silhouette score - Cross-validation (K-fold)

In-Class: - Clustering visualizations - Elbow method walkthrough - Cross-validation strategy discussion - Algorithm selection decision trees

Self-Study: - Cluster datasets (find optimal K) - Apply PCA to high-dimensional data - Use cross-validation in your models - Compare algorithms on same dataset

SUCCESS METRIC: Students can explain when to use K-Means vs. Hierarchical vs. DBSCAN.

Week 11: Deep Learning Part 1 — Neural Networks & PyTorch (5-6 hours)

Notebooks: - intro_DL1.qmd (2.5 hours) — Neural network basics, backpropagation, layers - intro_DL2.qmd (2.5-3 hours) — PyTorch fundamentals, building networks

Learning Objectives: - Understand neural network architecture (layers, neurons, activations) - Grasp backpropagation intuitively (not mathematically) - Build neural networks in PyTorch - Train on GPU using Google Colab - Debug training (learning curves, convergence)

Key Concepts: - Neurons and layers - Activation functions (ReLU, Sigmoid, Softmax) - Forward pass and backpropagation - Loss functions - Optimizers (SGD, Adam) - GPU vs. CPU training

In-Class: - Neural network architecture visualization - GPU concepts explained (why it's faster) - PyTorch syntax walkthroughs - Google Colab setup demo - Learning curve interpretation

Self-Study: - Build 2-3 neural networks in PyTorch - Train on Google Colab (free GPU) - Interpret loss curves - Experiment with architectures (layers, units)

NOTE: This week moves fast. Deep Learning is abstract; students need hands-on repetition.

Week 12: Deep Learning Part 2 & Agentic AI Capstone (6-8 hours)

Notebooks: - `intro_DL3.qmd` (2 hours) — Advanced architectures (CNN, RNN), transfer learning - `opencode_student_guide.qmd` (2-3 hours) — Orchestration frameworks, LLMs, RAG, CodeTrain patterns - Capstone project (2-3 hours) — AI application combining deep learning or agents

Learning Objectives: - Understand CNN and RNN architectures - Apply transfer learning (use pre-trained models) - Understand LLMs as pattern-matching systems and RAG as a solution to hallucination - Learn the orchestration framework pattern for workflow design - Build interactive agents with modern frameworks (chatbots, researchers, analyzers) - Know when to use educational frameworks vs. production solutions - Build an AI application solving a real problem (deep learning model, agent, or hybrid) - Pitch your project to investors/stakeholders

Key Concepts: - Convolutional Neural Networks (image processing) - Recurrent Neural Networks (sequences, text) - Transfer learning workflow - Large Language Models and prompt engineering - Retrieval Augmented Generation (overcoming hallucination) - Orchestration frameworks and their ecosystem - Educational vs. production frameworks - Building stateful, interactive agents

In-Class: - CNN/RNN walkthroughs - Transfer learning demos - LLM and RAG conceptual walkthrough - Orchestration framework explanation - Live demo: building an interactive chatbot - Framework comparison (educational vs. production) - Capstone project presentations - Feedback and iteration

Self-Study: - Build CNN for image classification - Build RNN for sequence task - Use pre-trained model (ResNet, BERT, etc.) - Install and explore orchestration framework - Build at least one interactive agent (chatbot, researcher, or data analyzer) - Experiment with conditional routing and job chaining - **CAPSTONE:** Build portfolio project combining deep learning OR agentic components: - Define problem (Ghana context) - Gather data - Build model (neural network, agent, or hybrid) - Deploy or demo - Write technical documentation

SUCCESS METRIC: Each student has portfolio-ready project demonstrating full AI pipeline (deep learning + optional agentic components).

Summary Table

Week	Module	Topic	Hours	Critical Gate?
1	00-01	Welcome + Setup	4-5	No
2	02	Git/GitHub	3-4	No
3	03.1	Python Basics 1	4-5	YES

Week	Module	Topic	Hours	Critical Gate?
4	03.2	Python Basics 2	4-5	YES
5	03.3	Python Basics 3	4-5	Gate
6	04.1	Pandas 1	5-6	YES
7	04.2	Pandas 2	5-6	YES
8	04.3	Pandas 3	5-6	Gate
9	05.1-2	ML Part 1	5-6	Medium
10	05.3	ML Part 2	5-6	Medium
11	06.1-2	DL Part 1	5-6	YES
12	06.3+07	DL Part 2 + Capstone	6-8	Gate

Critical Gates

Gate 1 (Week 5): Python Mastery - If <80% understand comprehensions/functions, slow down - This is non-negotiable; everything after depends on it

Gate 2 (Week 8): Pandas Fluency - If <80% comfortable with groupby/merge, remediate - Can't do ML without Pandas

Gate 3 (Week 12): Capstone Completion - Portfolio project must be working and documented - This is job application material

Key Adjustments from Original Proposal

- Week 1: Combined Welcome + Setup (was Week 1-2)
 - Weeks 3-5: Python gets 3 weeks (was rushing through)
 - Weeks 6-8: Pandas gets 3 weeks (critical for 50% beginners)
 - Weeks 9-10: ML compressed from 3 to 2 weeks (students can handle faster pace after 8 weeks)
 - Weeks 11-12: DL compressed from 3 to 2 weeks (hands-on practice > lectures)
 - Agentic AI (Module 07) integrated into Week 12 capstone project
-

Self-Study Breakdown

Total self-study per week: 10-15 hours

- Weeks 1-2: 10-12 hours (setup, environment, practice)
 - Weeks 3-5: 12-15 hours (heavy on exercises, practice)
 - Weeks 6-8: 12-15 hours (data cleaning, transformations)
 - Weeks 9-10: 12-15 hours (model building, tuning)
 - Weeks 11-12: 15-18 hours (capstone project, deep work)
-

Trainer Responsibilities (Weekly)

In-Class (3-4 hours): - Live coding demonstrations - Answer student questions - Facilitate group exercises - Identify students struggling (gates)

Between Classes: - Grade exercises/assignments - Prepare next week's content - Create practice problems - Document common questions

This Guide Is Your Pacing Blueprint

Use this to answer trainer questions: - "Where should Week 5 students be?" → "Python comprehensions + functions, no Pandas yet" - "How much time for Pandas?" → "3 full weeks (Weeks 6-8)" - "When do students learn ML?" → "Weeks 9-10, after solid Pandas foundation" - "Is capstone enough?" → "Yes, if they're doing real data + real model + real deployment"